Spec Driven Development - Quick Start Guide

Welcome to Spec Driven Development (SDD)! This guide explains when and how to create Spec files for product features.

What is a Spec?

A **Spec** is a detailed implementation blueprint that lives in this Git repository. It's where PM, Design, and Engineering collaborate to document exactly HOW a feature will be built before implementation begins.

Epic vs Spec: What's the Difference?

We use **both** Epics and Specs - they serve different purposes:

Aspect	Epic (ProductBoard/Jira)	Spec (This Repo)
Owner	Product Manager	Collaborative (PM + Design + Eng)
Purpose	Business planning, prioritization, tracking	Implementation blueprint, AI-friendly context
Focus	WHAT & WHY	HOW
Audience	Stakeholders, leadership, roadmap planning	PM, Design, Engineering, AI assistants
Location	ProductBoard/Jira	Git repo (version-controlled with code)
Contains	User problem, business value, success metrics, high-level scope, team assignments, timeline	Detailed user flows, technical architecture, data models, API contracts, testing strategy
Detail Level	High-level ("Users can export data")	Specific ("Export button top-right, CSV/JSON, max 10k rows")
Updates	Throughout feature lifecycle	Locked at approval, updated if scope changes
Visibility	Business, product, engineering	Engineering, design, technical implementation
Examples	"Enable data export to increase retention by 15%"	"POST /api/exports, sync <1000 rows, async >1000 rows, S3 storage, email notification"

Think of it this way:

- **Epic** = "We need to let users export their data because it increases retention by 15%"
- **Spec** = "Export button in top-right nav, supports CSV/JSON, max 10k rows, uses streaming API, handles errors with toast notifications..."

When to Create a Spec

DO create a spec for:

- New user-facing features
- Significant technical work (new services, major refactors)
- Features involving PM, Design, AND Engineering
- Work estimated at 1-3 sprints

DON'T create a spec for:

- Bug fixes (unless architecturally significant)
- Minor copy changes or CSS tweaks
- Configuration changes
- Hotfixes or urgent patches

Not sure? Ask: "Will this require collaboration across PM/Design/Engineering to define the approach?" If yes, create a spec.

Spec Granularity

One Epic → One Spec (Most Common)

For straightforward features that can be implemented in 1-3 sprints.

Example: Epic "Add CSV Export" → Spec `2026-03-csv-export.md`

One Epic → Multiple Specs (Complex Work)

For large initiatives that break into natural phases.

Example: Epic "Revamp User Dashboard"

- Spec 1: `2026-03-dashboard-data-layer.md`
- Spec 2: `2026-04-dashboard-ui-components.md`
- Spec 3: `2026-05-dashboard-personalization.md`

Multiple Epics → One Spec (Rare)

For shared technical foundations supporting multiple business features.

Example: Specs for authentication service referenced by multiple feature Epics

Sizing Guideline: Can this spec be reviewed in one sitting (~20-30 min read)? If not, consider breaking it down.

The Workflow

```
1. PM creates Epic in ProductBoard/Jira
   ↓
2. PM creates Spec file using TEMPLATE.md
   PM fills: Context & Background, initial Acceptance Criteria
   (Sources information from Epic - link and summarize, don't duplicate)
```

```
   Status: Draft
   ↓
3. Designer fills: User Experience section
   Designer refines: Acceptance Criteria (adds accessibility, interaction
details)
   Status: Still Draft
   ↓
4. Engineering fills: Technical Approach, Testing Strategy
   Engineering refines: Acceptance Criteria (adds technical details)
   Status: Still Draft
   ↓
5. Team Review: Resolve Open Questions, all roles sign off
   Status: In Review → Approved
   ↓
6. Implementation begins (Epic status: In Progress)
   Engineers reference Spec during development
   ↓
7. Feature complete
   Epic status: Done
   Spec status: Implemented
```

Golden Rule: No coding starts until Spec status = "Approved"

Section Ownership

Each section of the Spec has a clear owner:

Section	Owner	Contributors
Context & Background	PM	-
User Experience	Design	PM (user perspective), Eng (constraints)
Technical Approach	Engineering	Design (constraints), PM (scope)
Acceptance Criteria	Shared	PM drafts, Design & Eng refine
Testing Strategy	Engineering	PM (scenarios), Design (visual QA)
Implementation Plan	Engineering	-
Open Questions	Shared	Anyone can add, team resolves
References	Shared	All roles add relevant links

Owner = Responsible for drafting and maintaining that section **Contributors** = Provide input and review

Creating Your First Spec

1. **Copy the template**: `cp specs/TEMPLATE.md specs/YYYY-MM-DD-feature-name.md`
2. **Fill in your section** (based on your role)
3. **Commit and share** with team for their contributions
4. **Review together** and resolve Open Questions

5. **Mark as Approved** when all roles sign off
6. **Reference during implementation**

Keeping Epic and Spec in Sync

What lives where:

Information	Source of Truth
Business value, success metrics, user problem	Epic
Detailed user flows, UI decisions, technical architecture	Spec
High-level scope	Epic
Detailed acceptance criteria	Spec
Team assignments, timeline	Epic
Status tracking	Epic

When scope changes during implementation:

- **Small changes**: Update Spec inline, note in commit message
- **Significant changes**: Update Epic first (impacts roadmap), then update Spec, team re-reviews

After implementation:

- Epic status → "Done" (tracks delivery)
- Spec status → "Implemented" (historical record)
- Both remain as documentation for future team members

Tips for Success

1. **Start simple**: Don't over-document. Answer the questions needed for implementation.
2. **Link, don't duplicate**: Reference Epic, Figma, Mural instead of copying content
3. **Collaborate early**: Get all three roles (PM/Design/Eng) involved before approval
4. **Keep it current**: If scope changes significantly, update the Spec
5. **Use it during dev**: Reference the Spec when questions arise instead of asking in Slack
6. **Retrospect**: After each feature, discuss what worked and what didn't

Resources

- **Full Design Document**: `docs/superpowers/specs/2026-03-24-spec-driven-development-design.md`
- **Spec Template**: `specs/TEMPLATE.md`

Additional Resources

- **Spec Status Tracking**: `.spec-status-guide.md` - How to update and track spec statuses
- **Example Spec**: `EXAMPLE-2026-03-csv-export.md` - Reference example for pilot team

- **AI Assistant Integration**: `../docs/ai-assistant-integration.md` - Using specs with AI coding assistants

Questions?

This is a pilot framework! As you use it, we'll refine based on what works and what doesn't. Share feedback with your PM, Design, and Engineering partners.

Remember: The goal isn't perfect documentation - it's shared understanding before implementation begins.

Navigation

Getting Started: Team README · Spec Template · Example Spec

Guides: Status Tracking · AI Integration · Pilot Discussion

Framework: Design Document · CLAUDE.md